

Design the Architecture of a Machine Learning System

Project: Industry AI Flow – AI-Powered Construction Industry Platform

Program: Integrated Artificial Intelligence, SAIT Capstone Project

Team Members: - Angel Daniel Bustamante Perez - Jason Niu - Jack Si

Instructor: Reeta

Date: March 2026

Part A: High-Level Architecture

Our project, Industry AI Flow, is an AI-powered platform designed to assist the construction industry. We built three core capabilities: RAG-based document Q&A (so users can ask questions about construction documents and get cited answers), ML-driven cost estimation (predicting project cost overruns), and dynamic data analysis (generating and running code to analyze user-uploaded datasets).

A1. Data Sources

Source	Format	Purpose
Construction documents	PDF, images, CSV	RAG knowledge base (~12 industry documents)
Construction cost dataset	CSV (10,000 rows)	Training data for cost overrun prediction
User-uploaded datasets	CSV, Excel	Dynamic data analysis (ad-hoc exploration)
User chat input	Text (REST API)	Queries, cost estimation requests

A2. Data Storage

Storage	Technology	Content
Primary database	PostgreSQL 16 + pgvector	Document embeddings (768-dim vectors), conversation memory, audit logs, prompt versions
Vector index	IVFFlat (pgvector)	Accelerated similarity search on document chunk embeddings
Model artifacts	JSON on filesystem	Trained Ridge regression model weights, normalization parameters, evaluation metrics

Storage	Technology	Content
Document store	Filesystem + DB metadata	Raw uploaded PDFs, images, CSVs with extracted text chunks

A3. Data Processing (ETL)

We follow a standard ETL (Extract, Transform, Load) process for both the RAG pipeline and the cost estimation model.

Extract: Users upload documents through our web UI. We use PaddleOCR to extract text from images and scanned PDFs. CSV files are parsed directly with Python.

Transform: - For RAG: text is split into 512-character chunks with 128-character overlap, then each chunk is embedded into a 768-dimensional vector using nomic-embed-text-v1.5 (a local embedding model) - For cost estimation: we standardize numeric features using z-score normalization and apply one-hot encoding to categorical features (project_type, location)

Load: - Document chunks and their embeddings are stored in PostgreSQL with the pgvector extension - A BM25 keyword index is built in-memory to support hybrid retrieval - The trained cost estimation model is serialized to a JSON artifact file

A4. Model Training

We use two ML components in our system:

1. **Cost Estimation (Ridge Regression):** This model predicts construction cost overruns using 15 numeric features and 2 categorical features. We trained it on 10,000 construction project records with 5-fold cross-validation, achieving $R^2 = 0.989$ on actual cost prediction. We chose Ridge Regression for its interpretability and fast inference speed.
2. **RAG Retrieval Pipeline:** This combines BM25 keyword matching and vector similarity search using Reciprocal Rank Fusion (RRF), followed by a BGE cross-encoder reranker. While not a traditional ML model, it is a tuned retrieval system that we evaluate through groundedness checks.

A5. Model Evaluation

Model	Metrics	Values
Cost Estimation (Ridge)	MAE, RMSE, MAPE, R^2	MAE: \$2.58M, RMSE: \$9.83M, MAPE: 6.18%, R^2 : 0.989
Cost Estimation baseline	Same metrics	MAE: \$4.06M, MAPE: 8.59%, R^2 : 0.975
RAG Retrieval	Top-K relevance, groundedness score	Groundedness threshold ≥ 0.8 (lexical check)

Our cost estimation model outperforms the baseline (simply using the estimated cost as the prediction) by 28% relative improvement in MAPE. This confirms that the ML model adds meaningful value over a naive approach.

A6. Deployment

Component	Deployment
Backend API	FastAPI + Uvicorn on local hardware (Mac Studio M1 Max or Windows + RTX 5060)
Frontend	Next.js production build, served locally
LLM inference	Ollama (Qwen3.5:4b) running locally with Metal GPU acceleration
Database	PostgreSQL 16 + pgvector in Docker container
Code sandbox	Docker containers for isolated Python execution

For our Capstone demo, the entire system runs on local hardware. We optionally use cloud LLM APIs (Zhipu GLM-4, Google Gemini) as fallback for code generation tasks, since local models are not strong enough for reliable code generation.

A7. Monitoring and Maintenance

Aspect	Implementation
API metrics	Prometheus counters: request latency, LLM call success rate, cache hit rate
Audit logging	Tenant-aware structured JSON logs for all API operations
Cost tracking	Per-tenant LLM usage logging with budget enforcement
Model versioning	Cost estimation model artifact includes training timestamp, dataset hash, and all hyperparameters
Health checks	FastAPI health endpoint; Ollama connectivity verification at startup

Part B: Detailed Architecture Diagram

Please see the two attached architecture diagrams (PDF). Both were created in draw.io.

We provide two complementary architecture diagrams:

Diagram 1: System Architecture (C4 Container Diagram)

This diagram shows the **technical system architecture** organized in six layers:

1. **Client Layer:** Next.js frontend with API proxy

2. **API Gateway:** FastAPI with JWT authentication, tenant isolation, and query caching
3. **Orchestration:** 11-node LangGraph intent classifier, routing decision engine, versioned prompt manager, 3-layer memory manager
4. **AI Runtime (3 paths):**
 - Path A (RAG): Embedding (nomic-v1.5) -> Hybrid Search (BM25 + vector + RRF) -> Rerank (BGE cross-encoder) -> LLM Generate (Qwen3.5) -> Groundedness Check
 - Path B (Cost Estimation): Feature Extract -> Ridge Regression (NumPy) -> Cost Report with Confidence Interval
 - Path C (Data Analysis): Metadata Extract -> Cloud LLM Code Gen (Gemini/Claude/GLM-4) -> Docker Sandbox -> Charts & Results
5. **Data & Storage:** PostgreSQL + pgvector (768-dim embeddings), Ollama LLM (local), cloud APIs (fallback), document store, audit logs
6. **Observability:** Prometheus metrics, Docker runtime

Data flow: The diagram shows how user queries flow from the browser through API validation, intent classification, and then route to one of three specialized processing paths. Each path accesses shared storage (PostgreSQL + pgvector) and LLM services (Ollama).

Interconnection: Arrows show synchronous REST calls (solid), async LLM inference (dashed), and database access (dotted).

Technologies: Every component is annotated with its specific technology (e.g., “FastAPI + uvicorn”, “nomic-embed-text-v1.5 768-dim”, “Qwen3.5:4b via Ollama”).

Diagram 2: Product Architecture

This diagram shows the system from the **user’s perspective**, illustrating the three user journeys:

- **Journey A (Blue):** User asks a question via Web Chat -> Intent Classification -> Hybrid Search -> Reranking -> LLM Generation -> Groundedness Check -> Cited Answer
- **Journey B (Purple):** User uploads a dataset -> Intent Classification -> Metadata Extraction -> Cloud LLM Code Generation -> Docker Sandbox Execution -> Charts & Reports
- **Journey C (Green):** User fills Cost Estimator Form -> Intent Classification -> Feature Extraction -> Ridge Regression -> Cost Prediction with Confidence Interval

The diagram also shows the **document ingestion pipeline**: PaddleOCR (PDF/CSV/Image) -> Chunking (512 char) -> Embedding (nomic-v1.5) -> pgvector storage.

At the bottom, the shared infrastructure layer shows the key technologies: Qwen 3.5 LLM (Ollama), pgvector DB, FastAPI + LangGraph backend, Next.js frontend, and Docker sandbox.

Part C: Data Pipeline

C1. Data Collection

Sources:

Source	Description
Construction industry documents	~12 PDF/image documents covering safety regulations, building codes, project management guides
Construction cost dataset	unified_construction_projects_enhanced.csv – 10,000 records with project type, sqft, floors, location, contractor rating, risk score, actual costs
User uploads	Ad-hoc CSV/Excel files uploaded through the web UI for dynamic analysis

Collection Methods: - Documents: Manual upload via web UI -> FastAPI endpoint -> stored on filesystem with metadata in PostgreSQL - Cost dataset: Pre-loaded CSV file provided by our team member Jack, who has construction industry background - User data: Real-time upload via REST API with file validation (type, size limits)

Frequency: - Documents: One-time bulk load, with incremental additions as needed - Cost dataset: Static training set (batch, one-time) - User uploads: On-demand (real-time, per user session)

C2. Data Processing

Cleaning: - Documents: PaddleOCR handles image-based text extraction; text normalization removes formatting artifacts - Cost dataset: Drop rows with missing target variables; convert numeric columns to float; remove records with zero/negative costs - All 10,000 training rows pass validation (no rows dropped)

Transformation: - Document text: Character-based chunking (512 chars, 128 overlap) preserves context across chunk boundaries - Embeddings: nomic-embed-text-v1.5 generates 768-dimensional vectors per chunk - Cost features: Z-score normalization for 15 numeric features; one-hot encoding for 2 categorical features (resulting in 42 total features) - Missing numeric values: Imputed with training data medians

Feature Engineering: - `cost_overrun_pct`: Primary prediction target (percentage over/under budget) - `actual_cost_cad`: Derived target calculated as `estimated_cost * (1 + overrun/100)` - Prediction intervals: Computed from Absolute Percentage Error (APE) quantiles at 50th, 80th, 90th, 95th percentiles - BM25 keyword index: Built from document chunk text for lexical matching alongside vector search

C3. Data Storage

Raw Data Storage: - Uploaded documents stored on the filesystem under `workspace/documents/` - Cost dataset CSV stored at `datasets/unified_construction_projects_enhanced.csv` - Metadata (filename, upload date, document type) tracked in PostgreSQL

Processed Data Storage: - Document chunks with embeddings stored in `document_chunks` table (PostgreSQL + pgvector) - Schema: `id`, `document_id`, `content`, `embedding vector(768)`, `metadata` - IVFFlat index on embedding column for fast approximate nearest neighbor search - Trained model artifact stored as JSON at `workspace/models/cost_estimation/latest.json` -

Contains: model weights (42 coefficients), bias, normalization means/stds, category levels, evaluation metrics - Conversation memory stored in `conversation_memories` table with optional vector embeddings for long-term retrieval

Part D: Model Training

D1. Model Selection

Algorithm: Ridge Regression (L2-regularized linear regression)

We chose Ridge Regression for the following reasons: - **Interpretability**: It produces directly interpretable feature weights, so we can explain which factors drive cost overruns (e.g., number of change orders is the strongest predictor) - **Efficiency**: It is lightweight enough for real-time predictions during our live demo - **Robustness**: L2 regularization helps prevent overfitting on our 42-feature space and handles multicollinearity from one-hot encoded categories - **Strong performance**: We achieved $R^2 = 0.989$ on actual cost prediction, which exceeded our expectations

Frameworks and Libraries: - NumPy: We implemented the Ridge regression from scratch using NumPy's linear algebra operations (`np.linalg.pinv`), without relying on scikit-learn for the model itself. This gave us a deeper understanding of the algorithm. - Python standard library: JSON serialization for saving the model artifact

D2. Training Process

Data Splitting: - 5-fold cross-validation with stratified fold splitting (random seed: 42) - Each fold: train on 8,000 rows, evaluate on 2,000 held-out rows - Out-of-fold predictions collected across all folds for unbiased evaluation - Final model: trained on all 10,000 rows after cross-validation confirms generalization quality

Hyperparameter Tuning: - Primary hyperparameter: `ridge_alpha = 10.0` (L2 regularization strength) - Alpha value selected to balance bias-variance tradeoff for the feature space size - Cross-validation folds ($k=5$) and random seed (42) are configurable via command-line arguments - We did not implement grid search or Bayesian optimization, as the model is simple enough that the default alpha performs well and further tuning showed diminishing returns

Training Environment: - Hardware: Mac Studio (Apple M1 Max, 32GB RAM) or Windows workstation (32GB RAM + RTX 5060) - Software: Python 3.13, NumPy, custom training script (`scripts/utilities/train_cost_estimation_model.py`) - Training time: < 5 seconds on either machine (lightweight computation) - Reproducibility: All parameters, normalization statistics, and category mappings saved in the JSON model artifact

D3. Model Evaluation

Metrics:

Metric	Cross-Validation	Train Fit	Baseline (est. cost only)
MAE (CAD)	\$2.58M	\$2.57M	\$4.06M
RMSE (CAD)	\$9.83M	\$9.76M	\$14.67M

Metric	Cross-Validation	Train Fit	Baseline (est. cost only)
MAPE	6.18%	6.15%	8.59%
R^2	0.989	0.989	0.975

Validation Strategy: - 5-fold cross-validation ensures the model generalizes well to unseen data - Cross-validation and training metrics are nearly identical, confirming no overfitting - Baseline comparison: using `estimated_cost` directly as the prediction (no ML model)

Benchmarks: - Model beats baseline on all metrics (28% relative MAPE improvement) - Prediction intervals provided at multiple confidence levels: - 50th percentile: 5.22% error - 80th percentile: 9.72% error - 90th percentile: 12.68% error (default confidence level)

Top Feature Importances (by model weight magnitude):

Feature	Weight	Interpretation
num_change_orders	+6.03	Strongest overrun predictor
risk_score	+3.46	Higher risk -> higher overrun
num_subcontractors	-2.63	More coordination -> less overrun
team_experience_years	-1.48	Experience reduces overruns
estimated_cost_cad	+1.40	Larger projects tend to overrun more
contractor_rating	-1.28	Better contractors -> fewer overruns

Part E: Deployment Strategy

Infrastructure

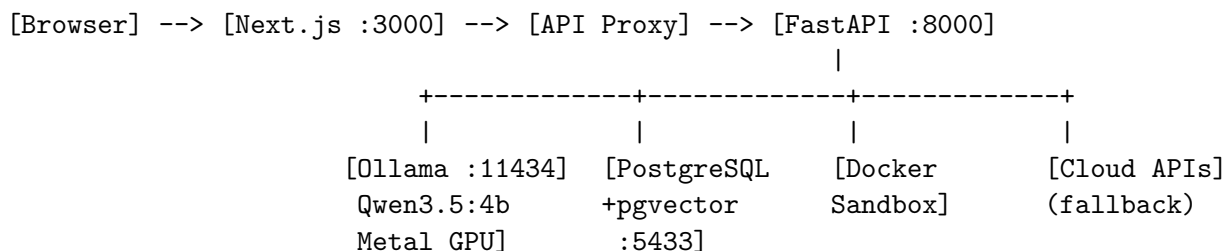
We deploy our system on **local hardware** for the Capstone demo. We designed it with containerization so that it could be scaled to cloud deployment in the future if needed.

Current Demo Deployment:

Component	Infrastructure	Details
FastAPI Backend	Local machine	Uvicorn ASGI server, port 8000, auto-reload in dev
Next.js Frontend	Local machine	Production build served on port 3000
PostgreSQL + pgvector	Docker container	pgvector/pgvector:pg16 on port 5433
Ollama LLM	Local machine	Qwen3.5:4b with Metal GPU acceleration (macOS)

Component	Infrastructure	Details
Code Sandbox	Docker containers	Isolated Python execution with resource limits

Deployment Architecture:



Containerization: - docker-compose-postgres.yml: PostgreSQL database with pgvector extension - Code execution sandbox: Docker containers with CPU/memory limits, network isolation, and execution timeouts - All services can be started with a single `make` command

Cloud-Ready Design: - Stateless API backend supports horizontal scaling behind a load balancer - Multi-tenant isolation (X-Tenant-ID) enables shared infrastructure - Environment-based configuration (`.env`) allows switching between local and cloud deployment - LLM dispatch supports hybrid mode: local Ollama for low-latency queries, cloud APIs (Zhipu, Gemini) for specialized tasks

CI/CD Quality Gates: - `make test-release-gate`: 11-gate pre-release validation - `make test-demo-smoke-gate`: CI-friendly smoke tests (no external dependencies) - Code quality: black formatter + mypy strict type checking + 70% test coverage minimum - 516+ unit tests with zero regressions

Security Measures: - JWT authentication with configurable API keys - Input sanitization to prevent injection attacks - Docker sandbox with restricted syscalls and no network access - Rate limiting per tenant - Audit logging for all operations